Mathematical Discoveries from Program Search with Large Language Models

Nature'23

Bernardino Romera-Paredes[1], Mohammadamin Barekatain[1], Alexander Novikov[1],Matej Balog[1], M.PawanKumar[1], Emilien Dupont[1], Francisco J. R. Ruiz[1],Jordan S. Ellenberg[2], Pengming Wang[1], Omar Fawzi[3], Pushmeet Kohli[1]& Alhussein Fawzi[1]

[1]Google DeepMind, London, UK.
[2]Department of Mathematics, University of Wisconsin-Madison, Madison, WI, USA.
[3]Laboratoire de l'Informatique du Parallélisme, University of Lyon (Inria, ENSLyon, UCBL, LIP), Lyon, France.

Google DeepMind

Presenter: 孙铂钛

2026.1.28

Content

- Background
- Design
- Experiments
- Conclusion
- Related works

背景与动机

LLM in Science

- LLM本身很难解决复杂问题，因为经常出现幻觉。
  - 👎 模型可能会生成看似合理但错误的输出。
  - 👍 相比于直接搜索解决方案本身，我们可以搜索生成解决方案的代码。

Bin Packing Problem

result? ✗

```python
def heuristic(item: float, bins: np.ndarray) -> np.ndarray:
  """Online bin packing heuristic discovered with FunSearch."""
  score = 1000 * np.ones(bins.shape)
  # Penalize bins with large capacities.
  score -= bins * (bins-item)
  # Extract index of bin with best fit.
  index = np.argmin(bins)
  # Scale score of best fit bin by item size.
  score[index] *= item
  # Penalize best fit bin if fit is not tight.
  score[index] -= (bins[index] - item)**4
  return score
```

algorithm? ✓

为什么是生成代码? (vs. Neural Networks)

- 可解释性：代码是白盒，领域专家可以理解并从中学习；NN是黑盒。
- 可扩展性：简洁的代码逻辑（如循环）天然可以扩展到更大规模的问题实例。

背景与动机

LLM in Science

- LLM本身很难解决复杂问题，因为经常出现幻觉。
  - 模型可能会生成看似合理但错误的输出。
  - 相比于直接搜索解决方案本身，我们可以搜索生成解决方案的代码

Motivation-1

- **LLM直接搜索解决方案会出现幻觉，我们可以搜索生成解决方案的代码**

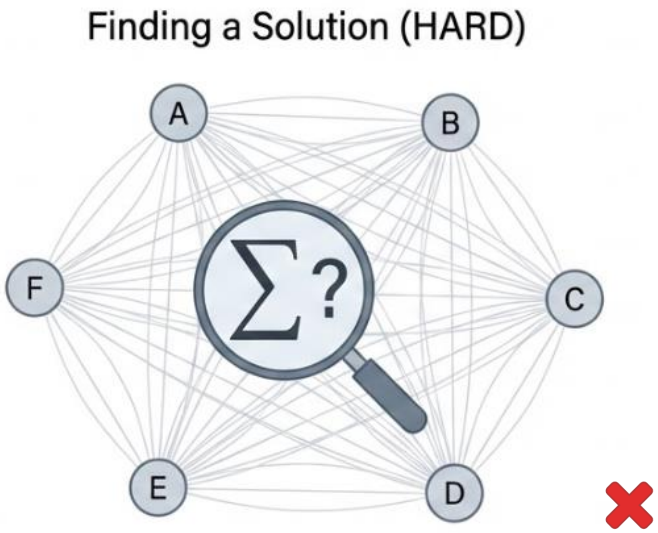
为什么是生成代码? (vs. Neural Networks)

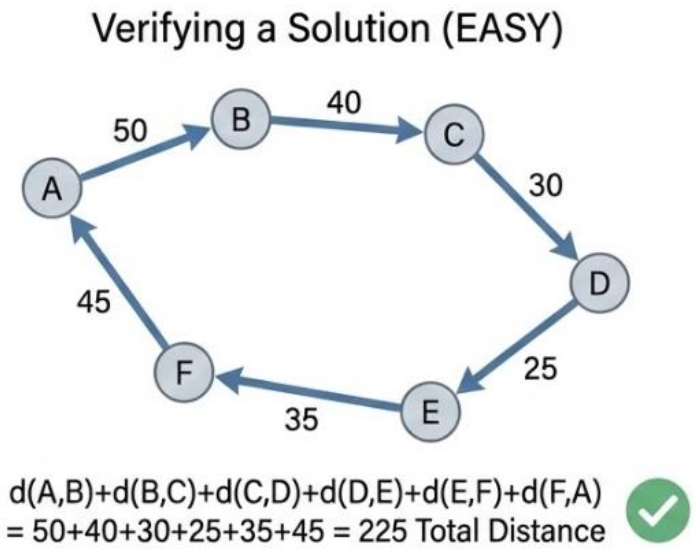
- 可解释性：代码是白盒，领域专家可以理解并从中学习；NN是黑盒。
- 可扩展性：简洁的代码逻辑（如循环）天然可以扩展到更大规模的问题实例。

背景与动机

许多科学与算法问题（尤其是NP类问题）中，验证一个解非常容易，但是找到一个解非常困难。

- 例如：旅行商问题中，找到最短路径很难，但是给定一条具体路径计算其距离很简单。





背景与动机

许多科学与算法问题（尤其是NP类问题）中，验证一个解非常容易，但是找到一个解非常困难。

- 例如：旅行商问题中，找到最短路径很难，但是给定一条具体路径计算其距离很简单。

Finding a Solution (HARD)

Verifying a Solution (EASY)

Motivation-2

- **NP问题中验证一个解容易，但是找到一个解很困难**

= 50+40+30+25+35+45 = 225 Total Distance

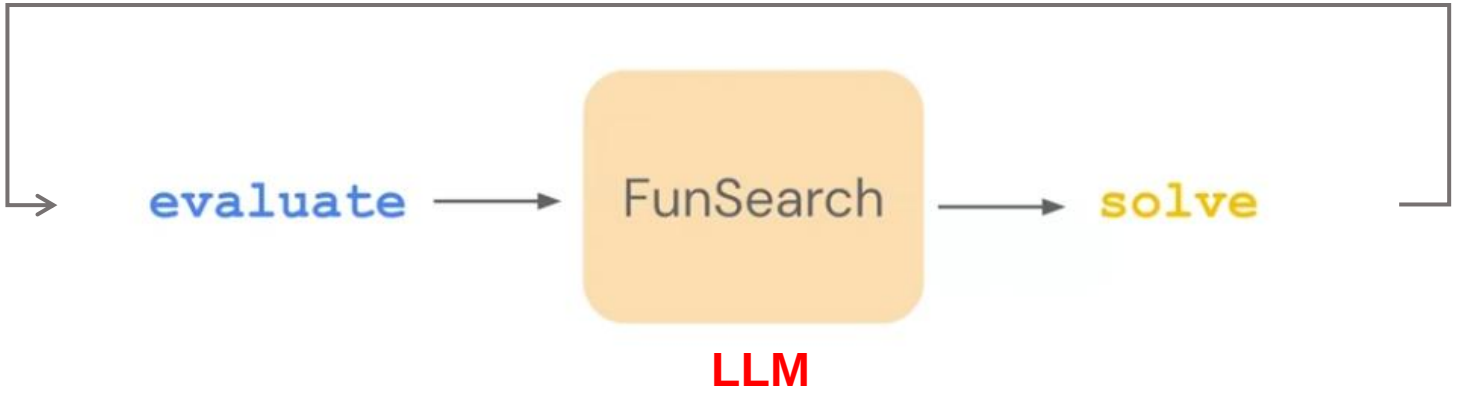
背景与动机

FunSearch (Searching in the Function Space)

- **能够针对某一个具体问题生成对应solution代码并动态进化。**
- **适用于直接求解难、但是验证解容易的问题，例如NP完全问题。**

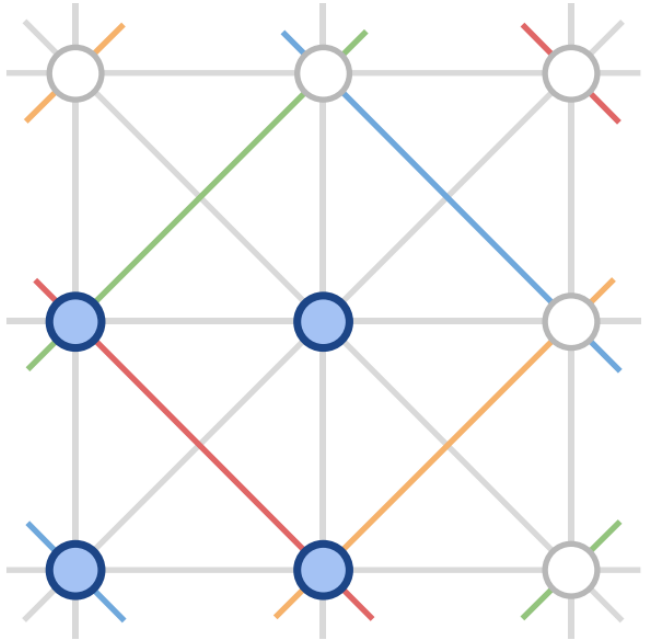
通过LLM + Evaluator实现迭代优化

- LLM作为生成器，提供创造性的思路和代码片段。
- Evaluator作为评估器，确保结果的正确性。



引例：极值组合学中的帽子集问题

- 定义：在 n 维空间 $\mathbb{Z}_3^n$ 中（$\{0,1,2\}^n$），**寻找最大的向量集合（cap set）**，使得**其中没有三个向量和为零（或者模3为0）**，几何直观上等同于没有三点共线。



$n = 2$时帽子集最大为4

当$n > 6$时，最大帽子集的大小是未知的，
我们能否发现更好的构造？

挑战：搜索空间随 n 指数级爆炸（$n = 8$时约为 3^{1600}），因此暴力搜索不可行！

引例：极值组合学中的帽子集问题

- 定义：在 n 维空间 $\mathbb{Z}_3^n$ 中，寻找最大的向量集合，使得其中没有三个向量和为零。

帽子集候选解的两个性质：

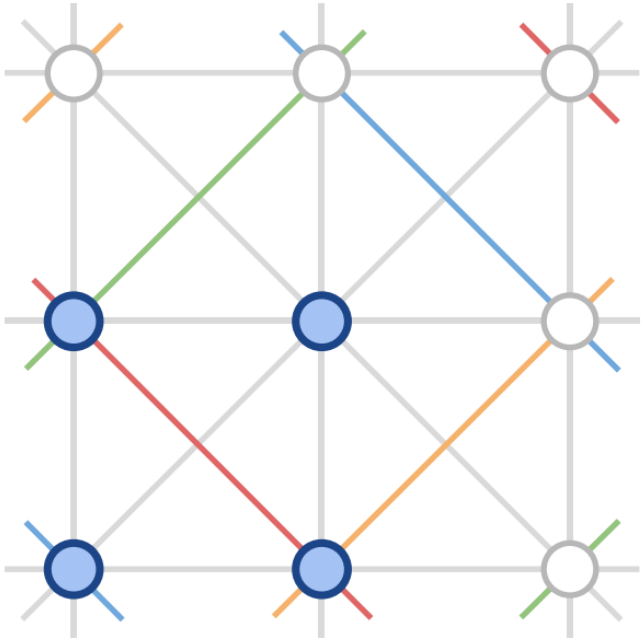
1、可以验证其正确性

- 该点集是否是一个帽子集？即，是否存在一条直线上包含超过三个点？

2、可以衡量其优劣

- 这个帽子集中包含多少个向量？

我们可以编写一个高效的evaluate函数来实现上述两方面。

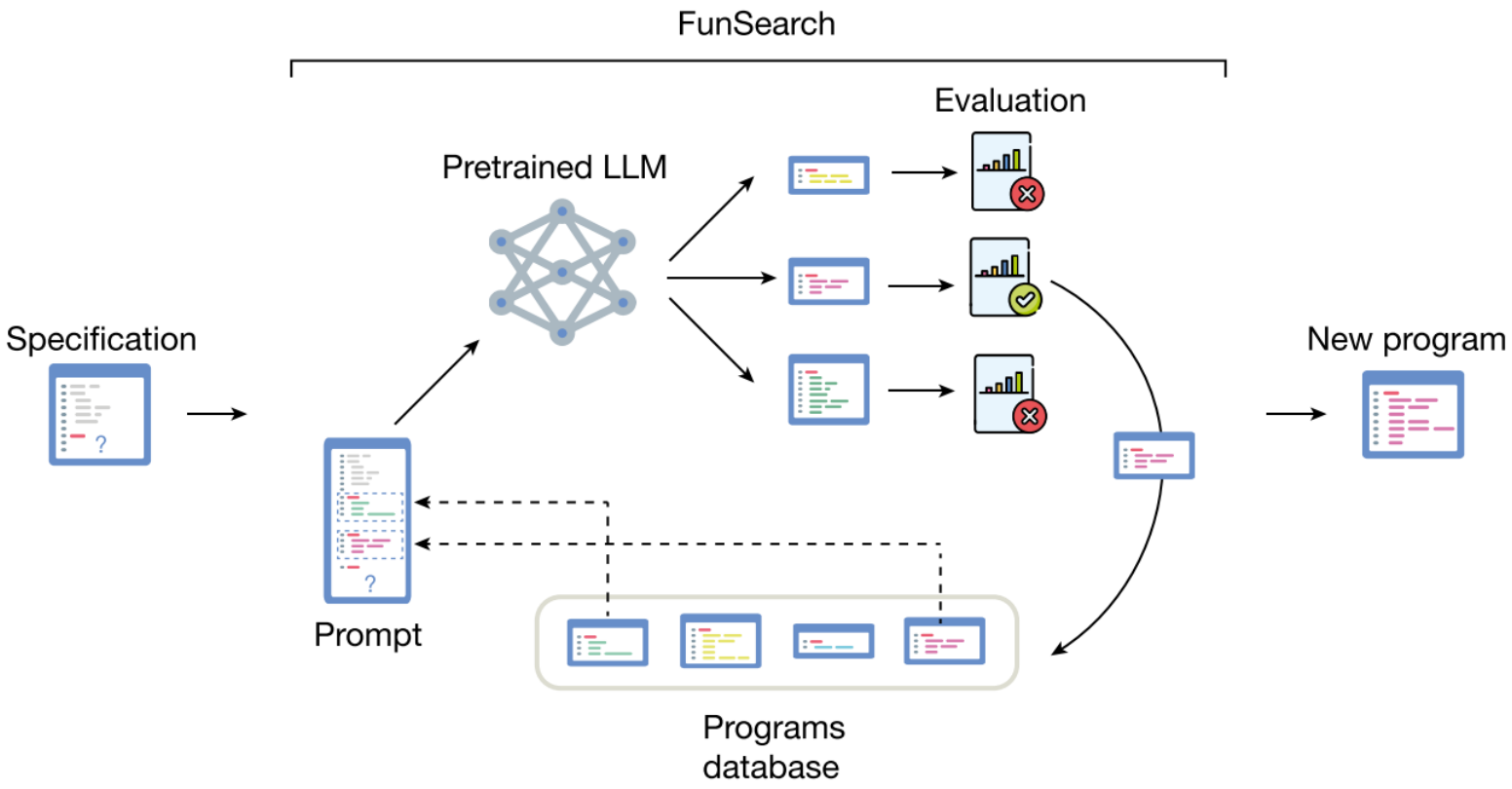


$n = 2$时帽子集最大为4

FunSearch overview

The Evolutionary Loop:

- Specification: 用户输入问题定义。
- Prompt: 从数据库采样程序构建提示词。
- LLM: 生成新程序。
- Evaluation: 运行并评分。
- Database: 保存正确且高分的程序 。



Design--Specification

FunSearch需要输入**代码骨架（skeleton）**：

- 包含样板代码和已知结构（例如这里的solve函数采用了贪心策略）。只让FunSearch进化控制核心逻辑的关键部分（例如这里每一步做出贪心决策的priority函数）。

目的：让LLM只专注于最难的算法逻辑，避免在基础语法上浪费算力或出错。

```python
"""Finds large cap sets."""
import numpy as np
import utils_capset

# Function to be executed by FunSearch.
def main(n):
  """Runs `solve` on `n`-dimensional cap set and
  ↪  evaluates the output."""
  solution = solve(n)
  return evaluate(solution, n)

def evaluate(candidate_set, n):
  """Returns size of candidate_set if it is a cap
  ↪  set, None otherwise."""
  if utils_capset.is_capset(candidate_set, n):
    return len(candidate_set)
  else:
    return None
```

```python
def solve(n):
  """Builds a cap set of dimension `n` using
  ↪  `priority` function."""
  # Precompute all priority scores.
  elements = utils_capset.get_all_elements(n)
  scores = [priority(el, n) for el in elements]
  # Sort elements according to the scores.
  elements = elements[np.argsort(scores,
  ↪  kind='stable')[::-1]]

  # Build `capset` greedily, using scores for
  ↪  prioritization.
  capset = []
  for element in elements:
    if utils_capset.can_be_added(element, capset):
      capset.append(element)
  return capset
```

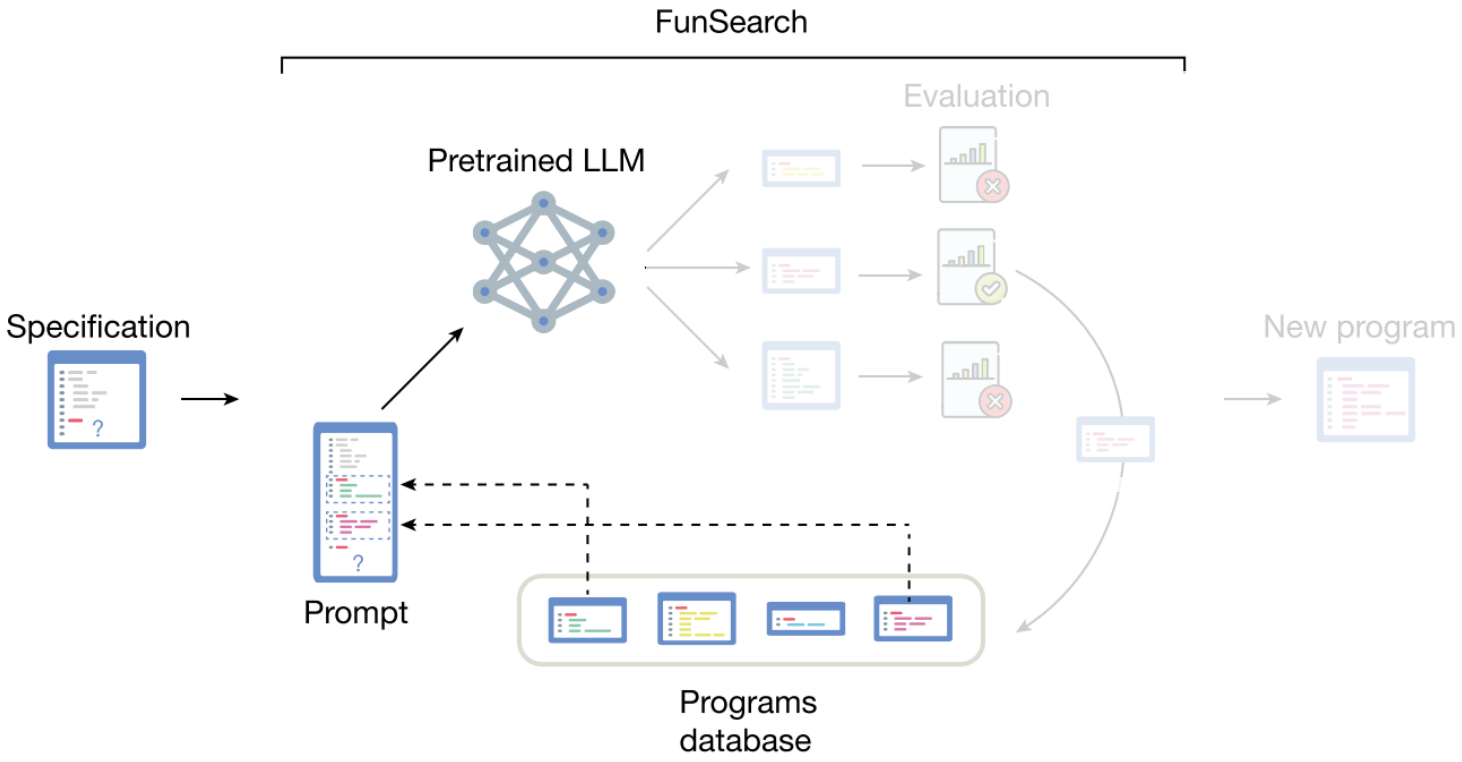
Evolve this function

```python
# Function to be evolved by FunSearch.
def priority(element, n):
  """Returns the priority with which we want to add
  ↪  `element` to the cap set."""
  return 0.0
```

Design--Prompt

如何让LLM进化代码?

- 从数据库中采样 k 个程序（通常 $k = 2$）。



Design--Prompt

如何让LLM进化代码?

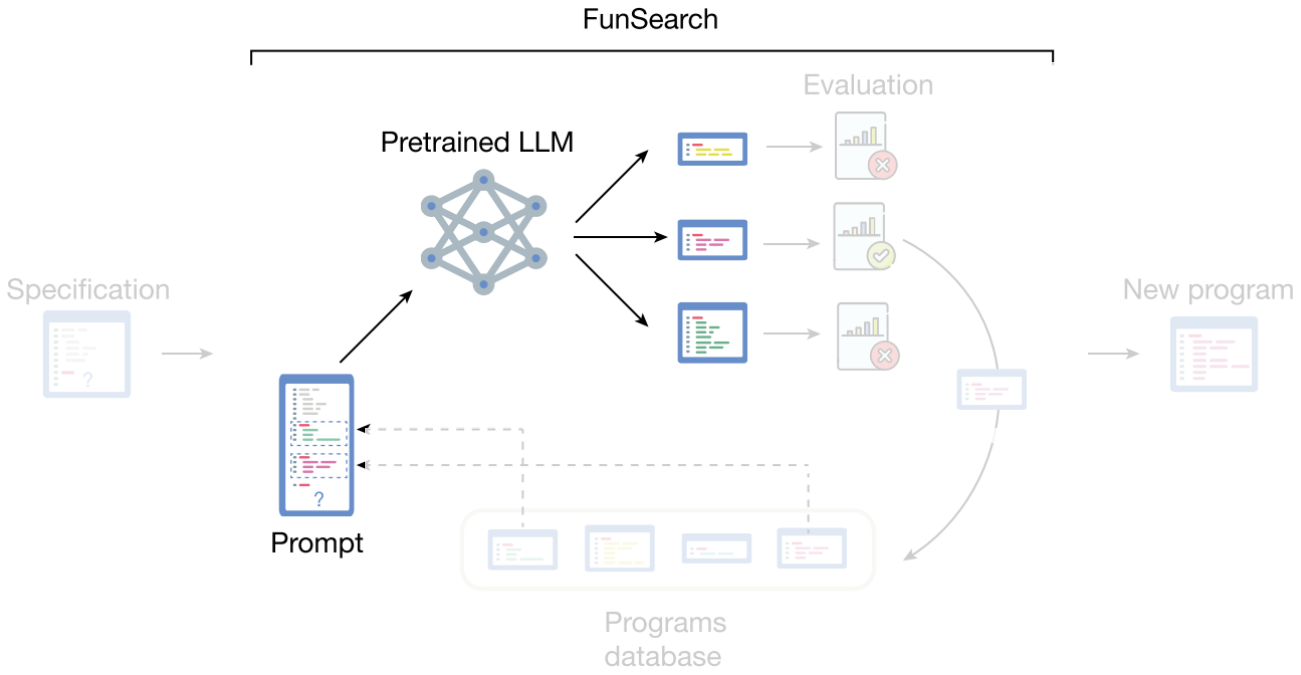
- 从数据库中采样 k 个程序（通常 $k = 2$）。
- 按分数排序，将版本作为函数名后缀：priority_v0 (低分)，priority_v1 (高分)。
- 将它们放入Prompt，给出priority_v2函数头，要求LLM生成priority_v2。

原理：利用LLM的上下文学习能力，**通过观察之前的版本，发现模式并泛化**，从而融合出更好的逻辑 。

```
"""Finds large cap sets."""
import numpy as np
import utils_capset

def priority_v0(element, n):
  """Returns the priority with which we want to add `element` to the cap set."""
  #######
  # Code from lowest-scoring sampled program.
  return ...
  #######

def priority_v1(element, n):
  """Improved version of `priority_v0`."""
  #######
  # Code from highest-scoring sampled program.
  return ...
  #######

def priority_v2(element, n):
  """Improved version of `priority_v1`."""
```

Design--Pretrained LLM

- **模型选择：**使用Codey (PaLM2 family)，在代码语料上微调过但没有在该问题上进行微调。
- **Trade-off：**
  - Quality vs. Speed: 选择了推理速度更快的模型而非高质量慢速的模型。
  - 逻辑：依靠进化算法的规模效应（采样数 ≈ 10^6）来弥补单次生成的质量不足 。
- **观察：**结果对具体LLM的选择不敏感，只要它受过足够的代码训练。

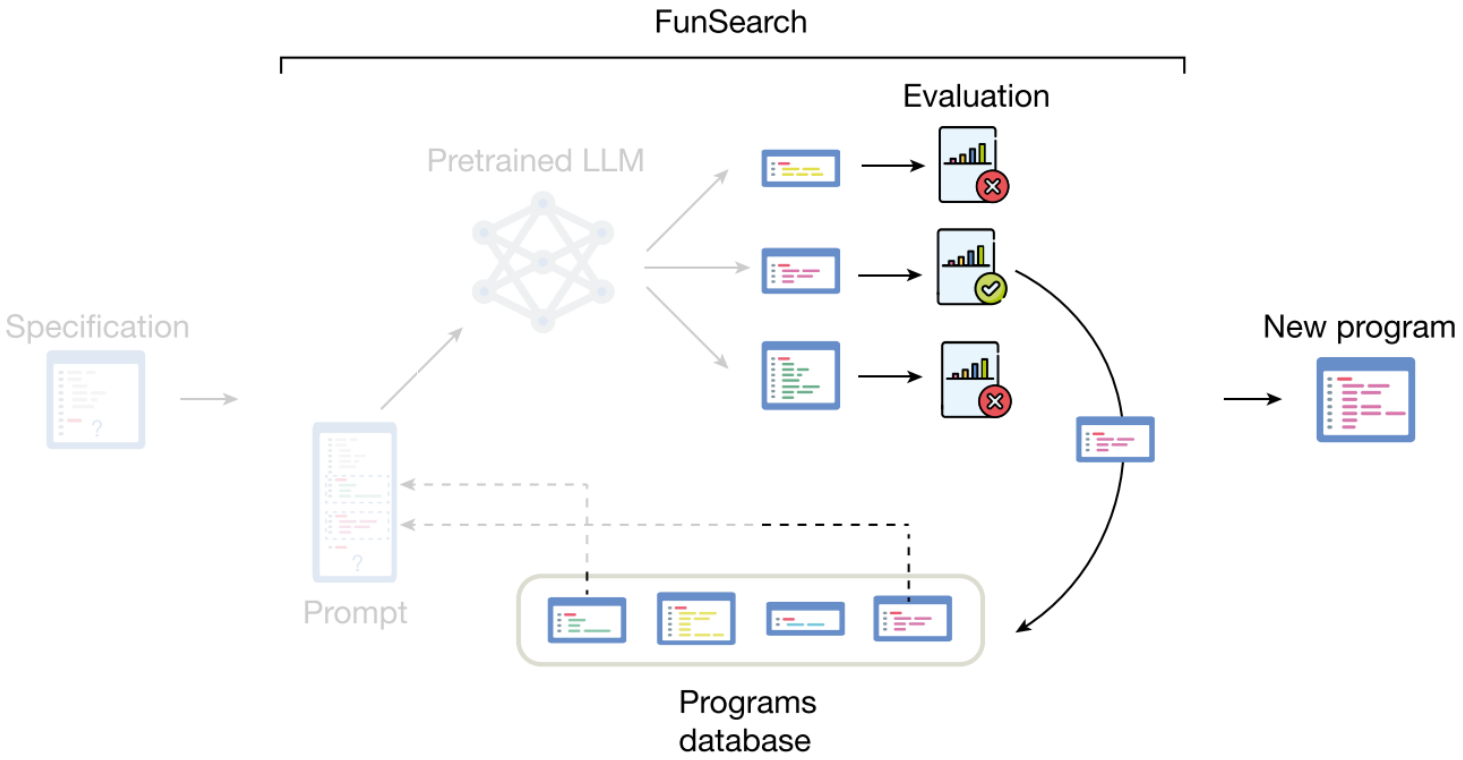


Design--Evaluation

Evaluator：用户提供，解决LLM的幻觉问题，确保存入数据库的程序都是验证过正确的

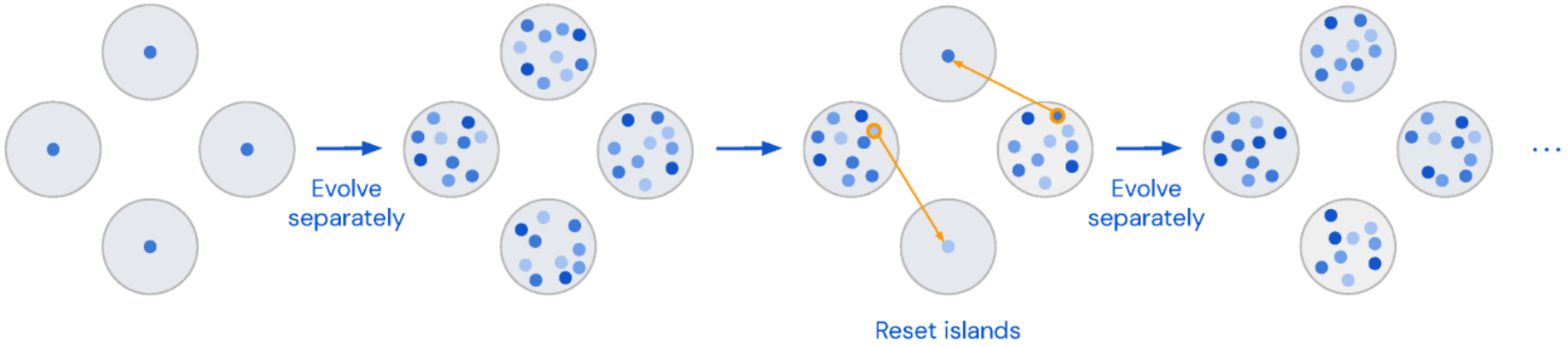
1、**Verify：**运行LLM生成的代码。如果出错或不满足问题的约束（如产生共线点），直接丢弃。

2、**Score：**根据优化目标对通过验证的程序打分（如帽子集的大小）。



Design--Programs Database (Islands Model)

- **目标：**保持多样性，鼓励探索避免陷入局部最优，允许并行进行实验。
- **Islands Model (岛屿模型)：**
  - 将种群分为多个独立的岛屿，独立进化。
  - 定期清除最差的一半岛屿，用存活岛屿中最好程序的克隆体重新填充。
- **采样：**先采样一个岛屿，再采样岛屿中的程序。倾向于**更高分**和**更短**的程序（越短的代码往往泛化能力越强）。



Design--Distributed Architecture

FunSearch天然并行，采样和评估可以并行执行，可以用分布式系统实现。

架构：异步通信的Workers。

- Programs Database：存储经过验证的代码。
- Samplers (15 nodes)：运行LLM (GPU/TPU)。
- Evaluators (150 nodes，5 CPUs)：运行评估 (CPU)。

优势：

- 并行性：采样和评估并行。
- 扩展性：允许利用大量廉价CPU解决评估耗时的问题。

实验1：帽子集问题

- 结果：在 $n = 8$ 维度上，FunSearch发现了大小为512的集合，超过了保持20年的已知最佳记录 (496) 。

Dimension *n*	2	3	4	5	6	7	8	...	n → ∞
Previous best construction	4	9	20	45	112	236	496	...	2.2180^n
***FunSearch* construction**	**4**	**9**	**20**	**45**	**112**	**236**	**512**	**...**	$\mathbf{2.2202^n}$

实验1：帽子集问题

- **可解释性：**通过人工审查FunSearch生成的代码，研究人员发现LLM利用对称性偏好“特定结构”的向量。

b

```python
def priority(el: tuple[int,...],
↪  n: int) -> float:
  score = n
  in_el = 0
  el_count = el.count(0)

  if el_count == 0:
    score += n**2
    if el[1] == el[-1]:
      score *= 1.5
    if el[2] == el[-2]:
      score *= 1.5
    if el[3] == el[-3]:
      score *= 1.5
  else:
    if el[1] == el[-1]:
      score *= 0.5
    if el[2] == el[-2]:
      score *= 0.5

  for e in el:
    if e == 0:
      if in_el == 0:
        score *= n * 0.5
      elif in_el == el_count - 1:
        score *= 0.5
      else:
        score *= n * 0.5 ** in_el
      in_el += 1
    else:
      score += 1

  if el[1] == el[-1]:
    score *= 1.5
  if el[2] == el[-2]:
    score *= 1.5

  return score
```

c

```python
def build_512_cap() -> list[tuple[int,...]]:
  """Returns a cap set of size 512 in `n=8` dimensions."""
  n = 8
  V = np.array(list(itertools.product(range(3), repeat=n)), dtype=np.int32)
  support = lambda v: tuple(i for i in range(n) if v[i] !=0)
  reflections = lambda v:sum (1 for i in range(1, n// 2) if v[i] == v[-i])

  # Add all 128 weight-8 vectors that have >= 2 reflections.
  weight8_vectors = [v for v in V
                     if np.count_nonzero(v) == 8  # Weight is 8.
                     and reflections(v) >= 2]  # At least 2 reflections.

  # Add all 128 weight-4 vectors that have specific support.
  supports_16 = [(0, 1, 2, 3), (0, 1, 2, 5), (0, 3, 6, 7), (0, 5, 6, 7),
                 (1, 3, 4, 6), (1, 4, 5, 6), (2, 3, 4, 7), (2, 4, 5, 7)]
  weight4_vectors = [v for v in V
                     if support(v) in supports_16]

  # Add all 128 weight-4 vectors with specific support and 1 reflection.
  supports_8 = [(0, 1, 2, 7), (0, 1, 2, 6), (0, 1, 3, 7), (0, 1, 6, 7),
                (0, 1, 5, 7), (0, 2, 3, 6), (0, 2, 6, 7), (0, 2, 5, 6),
                (1, 2, 4, 7), (1, 2, 4, 6), (1, 3, 4, 7), (1, 4, 6, 7),
                (1, 4, 5, 7), (2, 3, 4, 6), (2, 4, 6, 7), (2, 4, 5, 6)]
  weight4_vectors_2 = [v for v in V
                       if support(v) in supports_8
                       and reflections(v) == 1]  # Exactly 1 reflection.

  # Add 128 weight-5 vectors with <= 1 reflections and one more condition.
  allowed_zeros = [(0, 4, 7), (0, 2, 4), (0, 1, 4), (0, 4, 6),
                   (1, 2, 6), (2, 6, 7), (1, 2, 7), (1, 6, 7)]
  weight5_vectors = [
      v for v in V
      if tuple(i for i in range(n) if v[i] == 0) in allowed_zeros
      and reflections(v) <= 1  # At most 1 reflection.
      and (v[1] * v[7]) % 3 != 1 and (v[2] * v[6]) % 3 != 1]

  return weight8_vectors + weight4_vectors + weight4_vectors_2 +
  ↪  weight5_vectors
```

研究人员提炼的reflections洞察，进一步缩小搜索空间

LLM生成的代码

实验1：帽子集问题

利用这一洞察，研究人员修改了给FunSearch的Skeleton，强制要求生成的程序必须遵循这种对称性规则，减小了搜索空间，

- 引导FunSearch跑到了$n = 24$，发现了更大的容许集。
- 将帽子集容量的下界刷新到了 2.2202（$n \to \infty$时帽子集大小的增长率）。

```
def priority (el: tuple[int, ...], n: int, w: int) -> float:
  score = 0.0
  for i in range(n):
    if el[i] == 1:
      score -= 0.9 ** (i % 4)
    if el[i] == 2:
      score -= 0.98 ** (30 - (i % 4))
    if el[i] == 1 and el[i - 4] == 1:
      score -= 0.98 ** (30 - (i % 4))
    if el[i] == 2 and el[i - 4] != 0:
      score -= 0.98 ** (30 - (i % 4))
    if el[i] == 2 and el[i - 4] == 1 and el[i - 8] == 2:
      score -= 0.98 ** (30 - (i % 4))
      score -= 6.3
    if el[i] == 2 and el[i - 4] == 2 and el[i - 8] == 1:
      score -= 0.98 ** (30 - (i % 4))
    if el[i] == 2 and el[i - 4] == 1 and el[i - 8] == 1:
      score -= 6.3
    if el[i] == 2 and el[i - 4] == 0 and el[i - 8] == 2:
      score -= 6.3
    if el[i] == 1 and el[i - 4] == 1 and el[i - 8] == 0:
      score -= 2.2
  return score
```

improved capacity

insight (symmetry)

Capacity	Source
2.2101	Calderbank & Fishburn [1994]
2.2173	Edel [2004]
2.2180	Tyrrell [2023]
2.2184	FunSearch
2.2202	FunSearch (+ its insight)

实验2：在线装箱问题

物品在线装箱：给定足量、相同大小的箱子，以及一个不断到达的物品流，要求在线决策将每个物品放入某个箱子中，使得所使用的箱子总数最小。

- 结果：
  - 在OR-Library和Weibull benchmarks上均显著优于传统算法First Fit&Best Fit。
  - AI学会了一种策略——只有当物品能“完美填充”时才使用Best Fit，否则保留空隙给后续物品，避免产生无法利用的微小碎片。

	OR1	OR2	OR3	OR4	Weibull 5k	Weibull 10k	Weibull 100k
First fit	6.42%	6.45%	5.74%	5.23%	4.23%	4.20%	4.00%
Best fit	5.81%	6.06%	5.37%	4.94%	3.98%	3.90%	3.79%
FunSearch	**5.30%**	**4.19%**	**3.11%**	**2.47%**	**0.68%**	**0.32%**	**0.03%**

Fraction of excess bins (lower is better) for various bin packing heuristics on the OR and Weibull datasets. FunSearch outperforms first fit and best fit across problems and instance sizes.

```python
def heuristic(item: float, bins: np.ndarray) -> np.ndarray:
  """Online bin packing heuristic discovered with FunSearch."""
  score = 1000 * np.ones(bins.shape)
  # Penalize bins with large capacities.
  score -= bins * (bins-item)
  # Extract index of bin with best fit.
  index = np.argmin(bins)
  # Scale score of best fit bin by item size.
  score[index] *= item
  # Penalize best fit bin if fit is not tight.
  score[index] -= (bins[index] - item)**4
  return score
```

讨论

什么问题适合FunSearch?

1、Well-defined evaluator

- 必须拥有一个快速且明确的Evaluate函数来评估解的质量。

2、Rich scoring feedback

- 解的质量可以被量化（好一点or差一点），不能只是二元的True/False。
- 微小的改进能带来可测量的分数提升，使得可以通过进化的方式逐步优化。

3、Structured skeleton

- 需要能够将问题分解，隔离出关键逻辑供LLM进化。

局限性：

1、实验成本相对较高（寻找$n = 8$最大帽子集大约需要一百万次LLM采样）。

2、目前仅限于生成较短的Python函数。

3、需要一个快速的评估函数。

4、使用更强的LLM并未带来性能提升，这表明FunSearch并未充分利用LLM的推理能力。

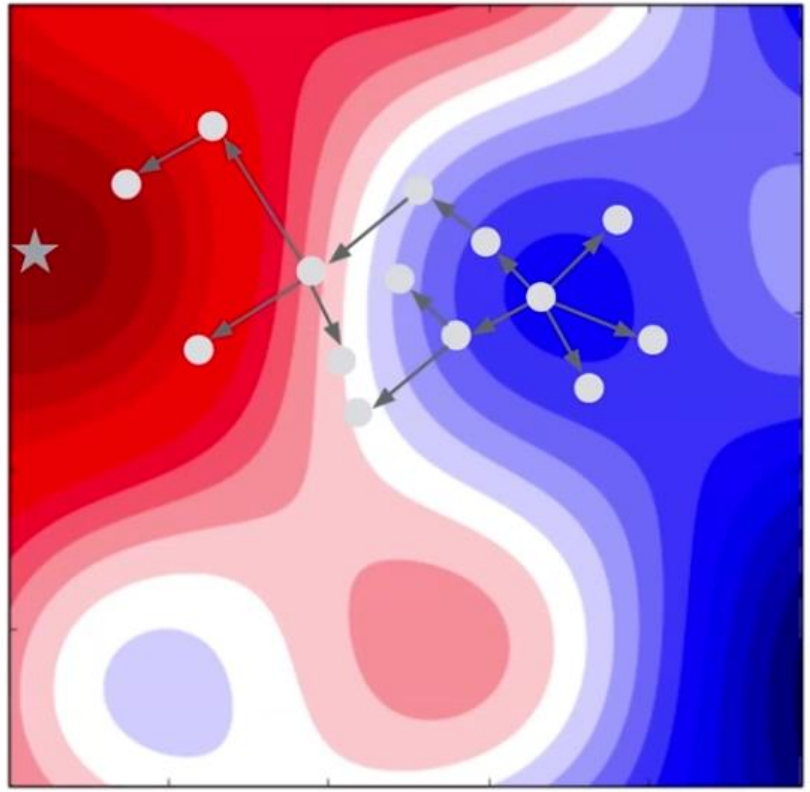
5、固定的skeleton会限制可能的程序空间。

Takeaways

- **搜索程序可以实现更高效的发现：**相比于直接搜索解决方案，**搜索描述如何解决问题的函数可以提供更结构化、更易于解释的表示方法。**并且许多科学问题都具有高度结构性而非随机性。与其它表示方法相比，用计算机程序描述解决方案通常更为简洁。
- **压缩程序长度可以实现可扩展性：**FunSearch倾向于简洁的程序，因此**与传统搜索方法相比，它可以处理规模更大的问题实例。**（FunSearch优化的是低Kolmogorov complexity，即产生给定输出的最短程序，而传统搜索算法则遵循不同的归纳偏置。）
- **LLM应被视为程序生成器，而非问题求解器：**预训练的LLM并不严重依赖于特定问题的上下文。相反，它能够生成多样化的、语法正确的程序，**偶尔会产生新颖且有用的想法。**
- **LLM天然偏爱人类编写的代码：**与遗传算法进化解决方案时没有任何人类可读性的偏好不同，LLM本质上偏爱结构化的、类似人类编写的代码，**这使得它们的输出更易于解释**，并且符合传统的编程实践。

总结

总的来说，FunSearch类似于进化算法，鼓励LLM去产生大量具有一定随机性的代码（变异），并使用evaluator去控制代码演化的方向。



Related Work

1、LLMs for Code & Discovery

- **程序化脚手架 (Programmatic Scaffold)：**研究者将LLM核心包裹在“程序化脚手架”中，**通过连接多个LLM调用完成更复杂的任务**，常结合外部工具或记忆流（如Toolformer、Generative Agents等系统）。
- **结合评估器 (LLM + Evaluator)：对LLM自身生成的数据进行微调**（如谜题解答或数学证明，以及问题答案的理由或解释），并使用评估器评估数据的正确性，确保微调数据集仅包含正确的解答或解释。
- **作为进化算子 (LLM as Mutation/Crossover)：**近期研究开始利用**LLM作为进化算法中的变异或交叉算子**，应用于生成虚拟机器人控制代码、神经网络架构搜索 (NAS) 以及Minecraft游戏探索等任务。
- **自动定理证明 (Automated Theorem Proving)：**利用LLM指导形式化证明的搜索，辅助自动定理证明器。

2、Genetic Programming (GP, 遗传规划)

- **程序自动发现：**遗传规划致力于通过进化方法自动生成或发现计算机程序，广泛应用于符号回归和优化算法的发现。在传统GP中，研究者必须手动定义“如何修改代码”，需要极强的领域知识。
- **超启发式 (Hyper-heuristics)：**专注于为组合优化问题（如装箱问题）学习启发式算法。这些方法**通常在特定问题实例上进化启发式策略**，使其性能能够匹敌传统的人工设计策略（如First Fit/Best Fit），但**需要手动定义启发式组件并针对具体问题调优。**
- **GP + LLM：**将LLM与遗传规划结合，**视LLM为一种高级的变异和交叉算子**，这被视为遗传规划的一种新范式。

3、Program Superoptimization (程序超优化)

- **语义保持优化 (Semantic-preserving Optimization)：**旨在修改程序代码以提高执行效率，同时保持其输入输出行为不变。
- **机器学习方法的引入：**利用强化学习 (RL) 或神经网络作为变异算子，优化汇编代码或特定算法；近期工作利用RL发现了更快的矩阵乘法和排序算法；利用LLM直接寻找C++或Python代码中提升性能的编辑方案。

Related Work

Large Language Models as Optimizers. (OPRO) ICLR’24

- 利用LLM上下文理解的能力，通过在提示词中包含“过往解决方案及其评分”的轨迹，引导LLM像进化算法一样迭代地生成更优质的解决方案（即让LLM自己优化提示词或求解数学问题）。

Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model.(EoH) ICML’24

- 通过让LLM在进化框架中同步进化启发式的自然语言思想与可执行代码，并设计针对性prompt策略引导LLM模仿人类专家的启发式开发过程，实现高效自动化的算法设计。

ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. (ReEvo) NeurIPS’24

- 将LLM作为超启发式引擎，通过进化搜索与双重反思机制（短期比较分析+长期经验积累）的协同，在开放的启发式空间中高效探索高质量算法设计，其中LLM的反思能力为离散搜索空间提供了类梯度的引导信号。

Olympiad-level formal mathematical reasoning with reinforcement learning. (AlphaProof) Nature’24

- 将强化学习与形式化数学系统深度融合，通过在数百万自动形式化问题上进行自我对弈式训练，并利用Test-Time RL在推理时针对难题动态生成并学习相关变体，使智能体能在严格可验证的环境中自主发现复杂数学证明。

Related Work

AlphaEvolve: A coding agent for scientific and algorithmic discovery. (AlphaEvolve) Nature’25

- 将LLM嵌入进化搜索循环，通过生成代码差异（而非完整程序）并让超参数本身参与进化，使智能体能在受控"幻觉"机制引导下自主发现数学构造与科学算法，实现从"编写代码"到"进化代码"的范式转变。

Multi-Objective Evolution of Heuristic Using Large Language Model. (MEoH) AAAI’25

- EoH框架的多目标扩展版本，将启发式设计扩展为多目标优化问题，通过在帕累托前沿上协同进化"自然语言思想"与"可执行代码"，使大语言模型能够自主发现兼顾解质量、计算效率等多维度性能的多样化启发式算法集合。

LLM4AD: A Platform for Algorithm Design with Large Language Model. (LLM4AD) ACM Computing Surveys’26

- 是一个面向算法自动设计的综合性研究平台与基准测试套件，旨在系统评估大语言模型在算法发现与优化任务上的能力。

Thinking

一、能不能用到我们的场景?

问题：MoE门控路由依赖于人工设计的启发式算法，可以用FunSearch来寻找更优的启发式算法，再从中总结规律；实现为特定MoE规模自动生成“负载感知路由策略”。

思考：有可量化的指标

- 质量指标：下游任务准确率（验证路由对模型性能的影响）
- 效率指标：专家负载标准差、通信延迟

Thinking

二、问题可以泛化吗?

问题：FunSearch依赖稠密评分信号（如bin packing的装箱数、cap set的集合大小），但许多场景仅有稀疏/二元反馈。

思考：将二元反馈解耦为多阶段稠密信号：

- 子目标分解：例如对证明任务，用中间状态构建学习信号。
- 当问题可分解为“局部可评估的子结构”时，就可以用FunSearch的思想解决。

Thinking

三、能不能进一步提高?

问题：FunSearch每次变异比较随机，缺乏对搜索空间的全局认知。

思考：引入语言梯度机制

- 在每次评估后，让LLM对“高分程序 vs 低分程序”进行对比反思。
- 将反思提炼为可操作的进化方向，注入下一轮prompt。
- 参考ReEvo的反思机制：短期反思+长期反思。

问题：skeleton设计依赖人工，限制了搜索空间的表达能力。

思考：skeleton自进化

- 同时维护两层种群：Skeleton Population（程序结构） + Logic Population（核心逻辑）
- 每N代允许skeleton种群发生“突变”（如从贪心策略切换到动态规划策略）
- 约束skeleton复杂度，避免过拟合

Q & A